# Explaining RSA Encryption

Yuval Atia

# Contents

This entry assumes undergraduate knowledge in group theory & elementary number theory, however all results and definitions that are used are presented in an orderly fashion, so aside from some group theoretic mumbo-jumbo which is necessary for the retrieval of some key result which we use later on, readers of high-school mathematical background should also be able to follow along for the most part.

## Introduction

Nowadays, online encryption is ubiquitous: since adversaries may eavesdrop on network traffic, it is imperative that confidential information be shared in a secure manner, such that only the trusted parties would be able to retrieve the raw data. Conceptually, all encryption schemes can be reduced to two functions: `encrypt` and `decrypt`, as laid out below:

```
def encrypt(secret: bytes, key: Key) -> bytes:
    encoded_secret = encode(secret)
    encrypted_secret = encryption_function(key, encoded_secret)
    return encrypted_secret

def decode(encrypted: bytes, key: Key) -> bytes:
    decrypted_secret = decryption_function(key, encrypted)
    plain_secret = decode(decrypted_secret)
    return plain_secret
```

Encoding and decoding schemes are of less relevance to us, essentially they are injective functions from the plain data space to the domain of the encryption function, and vice versa. For example, if the encryption scheme operates on integers and the plain data is text encoded in UTF-16, one possible encode/decode function would convert the textual representation to its numerical UTF-16 representation and vice versa (note that this also implies that data should be encrypted in chunks due to computational limitations when working with very large numbers, but that's irrelevant to our discussion). The heart of encryption schemes lies in two things: the keys, and the encryption/decryption functions.

Generally we consider two encryption schemes: symmetric and asymmetric. In a *symmetric encryption*, the same key is used for encryption and decryption, meaning it cannot be transmitted over an insecure medium, making it less useful for online communication. In an *asymmetric encryption*, different keys are used for encryption and decryption - the key used for encryption is called the *public key*, and can be safely transmitted over insecure medium, and the key used for decryption is called the *private key* and is kept secret. The public key itself is derived from the public key.

Since we assume that the medium over which the messages are transferred is not secure, i.e. adversaries may intercept and modify messages, it is imperative that adversaries who intercept encrypted messages would not be able to decrypt the messages, even if they are privy to the encryption scheme.

As a motivating example for this requirement, we consider perhaps the most well-known classic example of an encryption scheme - Caesar cipher, which can be thought of as a mapping $T : \mathbb{Z}_n^k \to \mathbb{Z}_n^k, x \mapsto x + cI$ where $c$ is some integer by which we shift all values. For example, given an encoding $a \mapsto 1, b \mapsto 2, \ldots, z \mapsto 26$, then $n = 26$ and given a key $c = 5$, the message "hello" is encoded to $(8, 5, 12, 12, 15)$, which under $T$ is mapped to $(8, 5, 12, 12, 15) + (5, 5, 5, 5, 5) = (13, 10, 17, 17, 20)$ (Which is decoded into "mjqqt"). This is an example of a *symmetric cipher*, since $c$ is used both as the

"encryption" key and as the decryption key, since evidently $T^{-1}$ is given by $x \mapsto x - cI$. It is common to see Ceaser cipher referred to as ROT$c$, where $c$ is the value by which we rotate the alphabet, in this case we considered ROT$5$. Note however that even without knowing $c$, it is incredibly easy to "break" the cipher - first, there are only ever $26$ possible preimages for every code $s$ since we use scalars from $\mathbb{Z}_{26}$, so given prior knowledge of the information being exchanged (here our knowledge is that the cipher is used to communicate a word in English), it is a very straightforward and simple task to consider all $26$ possibilities and see which of them are actual words in the English language. Also note that the *probabilistic properties* of the original letters is preserved, so one can consider the commonality of each latter in the ciphertext and make educated guesses as to what cipher was used. This type of attack can be mounted against more advanced "encryption" methods which rely on permutations.

We conclude that even if the encrypted message is intercepted, it should be difficult to inverse it without the key. It would also be nice for someone with the key to be able to quickly decrypt the message. This obviously applies to both symmetric and asymmetric encryption. The mathematical/computer science name for such a function is a *trapdoor function*, which we define roughly below:

**Definition 0.0.1** (trapdoor function)**.**
*A* trapdoor function *is a function that is easy to compute in one direction, but hard to compute in the other direction without special information.*

*More formally, if $\phi : A \to B$ is a trapdoor function then given $a \in A, \phi(a)$ is computationally easy to evaluate (which usually means "can be evaluated in polynomial time"), but given $b \in Im(\phi)$, it is computationally hard (i.e., cannot be evaluated in polynomial time) to find $\phi^{-1}(b) \in A$ s.t. $\phi(\phi^{-1}(b)) = b$, unless we know the special information, in which case finding this $\phi^{-1}(b) \in A$ becomes computationally easy.*

A trapdoor function may also be defined as a *one-way function* (functions whose image is computationally easy to calculate but the preimage of a given element in their image is computationally hard to calculate) with the additional property that provided certain information, the inverse becomes computationally easy to calculate.

Natural candidates for trapdoor functions are problems which we know to be solvable, but are hard to solve. One such example in number theory is the following:

**Definition 0.0.2** (integer factorization problem)**.**
*Given $n \in \mathbb{N}$, find primes $p_1, \ldots, p_k$ s.t. $n = \prod_{i=1}^{k} p_i$.*

We know that a unique solution exists for every $n$ by the fundamental theorem of arithmetic which we prove in elementary number theory, so the problem is solvable for every $n$, and while naive algorithms for finding this set of primes exist (most trivial of which is finding prime factors of $n$ that are less than or equal to $\lfloor \sqrt{n} \rfloor$ which can then be multiplied (with the right exponents) to produce $n$) the problem

in general is thought to be computationally hard in classical computing (it is worth mentioning that Shor's algorithm, which is a quantum computing algorithm, solves this problem in polynomial time).

Given the difficulty of the integer factorization problem, it can be used as the basis for a trapdoor function.

## Brief History of RSA

In 1977, Ron Rivest, Adi Shamir and Leonard Adleman published a paper describing their RSA encryption algorithm (*RSA* being an initialism of their last names). RSA makes use of modular arithmetic to provide an asymmetric encryption scheme, and the difficulty of the integer factorization problem is the basis for the trapdoor function used by RSA.

It is worth mentioning that an equivalent encryption scheme had already been developed earlier in 1973 by Clifford Cocks working for the GCHQ, however due to the secrecy of his work there, he could not publish his work. His work became declassified in 1997. Due to the state of computing at that time, the scheme was deemed too impractical, and was not used.

The three researchers credited for RSA were all working at MIT at the time, which patented RSA in 1983. However, since the paper describing RSA had been released on 1977, the patent had no legal standing outside the USA. On September of the year 2000, a few weeks before the expiration date of the patent, RSA security (which by then was the holder of the rights to the patent) released RSA to the public domain.

Over the next few sections, we will present relevant the mathematical background for RSA encryption, describe the algorithm and prove its correctness, and discuss some common attacks against RSA.

## Mathematical Background

In this section we use $\phi$ as Euler's totient function, which for every positive integer $n$ returns the count of positive integers less than $n$ that are relatively coprime to $n$ (i.e. $\phi(5) = 4, \phi(4) = 2$, etc.). In number theory and group theory, we have seen the following results which I will state without formal proof:

**Theorem 1** (basic properties of Euler's totient function)**.**
*1.* $\phi(n) = |\mathbb{Z}_n^\times|$
*2.* $\phi(p) = p - 1$ *if* $p$ *is prime*
*3.* $\phi(p^k) = p^k - p^{k-1}$ *is* $p$ *is prime.*

Proving $1$ is immediate from definition of $\mathbb{Z}_n^\times$ (the heavy-lifting there is in showing that $\mathbb{Z}_n^\times$ forms a group under modular multiplication $\mod n$). Proving $2$ is trivial - every prime number is divisible only by itself and $1$, so there are $p - 1$ coprimes w.r.t. $p$ that are smaller than $p$. Proving $3$ follows from a counting argument, where we show that there are $p^{k-1}$ multiples of $p$ in $[1, p^k]$, and from number theory we know by Euclid's lemma that if $p$ is prime that every number is either a multiple of $p$ or coprime to $p$.

We now work our way towards a much stronger result regarding Euler's totient function using tools from group theory.

**Theorem 2.**
*If* $M = \{m_1, \ldots, m_n\} \subset \mathbb{Z}$ *is a set of integers which are all pairwise relatively prime, then:*

$$\mathbb{Z}^\times_{\prod_{i=1}^n m_i} \cong \times_{i=1}^n \mathbb{Z}^\times_{m_i}$$

*Proof.* Denote $m = \prod_{i=1}^n m_i$ and consider the mapping

$$\psi : \mathbb{Z}_m^\times \to \times_{i=1}^n \mathbb{Z}^\times_{m_i},$$
$$[x]_m \mapsto ([x]_{m_1}, \ldots, [x]_{m_n})$$

We will show $\psi$ is an isomorphism.

Since $\psi$ is a mapping on equivalence classes, we must first show that $\psi$ is well defined. Suppose $x \equiv_m x'$, then $x = x' + zm$ for some integer $z$. We can also write $x' + zm$ as $x' + (z\frac{m}{m_i})m_i$, so we have $[x]_{m_i} = [x' + (z\frac{m}{m_i})m_i]_{m_i} = [x']_{m_i}$ for every integer $i$ in $[1, n]$, then $\psi([x]) = \psi([x'])$, then $\psi$ is well-defined.

Next, we show $\psi$ is a homomorphism. Let $[x], [y] \in \mathbb{Z}_m^\times$, then $[x][y] = [xy]$ by definition of modular multiplication, so

$$\psi([x]_m \cdot [y]_m) =$$
$$\psi([xy]_m) =$$
$$([xy]_{m_1}, \ldots, [xy]_{m_n}) =$$
$$([x]_{m_1}, \ldots, [x]_{m_n}) \cdot ([y]_{m_1}, \ldots, [y]_{m_n}) =$$
$$\psi([x]_m) \cdot \psi([y]_m)$$

So $\psi$ is a homomorphism.

It remains to show $\psi$ is bijective. Let $a = ([a_1]_{m_1}, \ldots, [a_n]_{m_n}) \in \times_{i=1}^{n} \mathbb{Z}_{m_i}^{\times}$, then $\psi$ is bijective if and only if $\exists![x] \in \mathbb{Z}_m^{\times}(\phi([x]_m) = a$, i.e. if the system of modular equations

$$x \equiv a_1 \mod m_1$$
$$\ldots$$
$$x \equiv a_n \mod m_n$$

Has a unique solution $\mod m$. By the Chinese remainder theorem, which we cover in Elementary Number Theory (which I have not published yet), we know that this is always the case, so $\psi$ is bijective.

We conclude that $\psi$ is an isomorphism, so $\mathbb{Z}_m^{\times} \cong \times_{i=1}^{n} \mathbb{Z}_{m_i}^{\times}$.

□

Since the heavy-lifting in this proof is done by the Chinese remainder theorem, in abstract algebra the chinese remainder theorem is often restated as follow

**Corollary 2.1** (extended Chinese remainder theorem)**.**
*Suppose $m_1, \ldots, m_n$ are relatively prime and let $m = \prod_{i=1}^{n} m$, then*

$$x \mod m \mapsto (x \mod m_1, \ldots, x \mod m_n)$$

*Defines an isomorphism*

$$\mathbb{Z}_m^{\times} \cong \mathbb{Z}_{m_1}^{\times} \times \cdots \times \mathbb{Z}_{m_n}^{\times}$$

Henceforth we sometimes refer to $\mathbb{Z}_n^{\times}$ as the *group of units* of $n$, denoted $U(n)$. This is merely a more concise notation that is easier to write. Furthermore, now that we no longer use $\mathbb{Z}$ in our notation of the group of units, we can refer to $\mathbb{Z}_n^{+}$ as simply $\mathbb{Z}_n$ without ambiguity. Groups of unit in general are a concept that comes from ring theory, which we will not discuss here.

The extended Chinese remainder theorem can be used to calculate $\phi(n)$, as demonstrated in the following example:

**Example 2.1** (finding $\phi(560)$)**.**
*Observe that* $560 = 7 \times 16 \times 5$*, all of which are relatively prime, so* $U(560) \cong U(7) \times U(16) \times U(5)$*, and* $16 = 2^4$*, so* $U(560) \cong U(7) \times U(2^4) \times U(5)$*. Since* $\phi(n) = |U(n)|$*, and the order of isomorphic groups is the same, and the order of an external direct product is the product of the orders of the groups in the product, then*

$$\phi(560) = |U(560)| =$$
$$|U(7) \times U(2^4) \times U(5)| =$$
$$|U(7)| \times |U(2^4)| \times |U(5)| =$$

$$\phi(7) \times \phi(2^4) \times \phi(5)$$

*Finally, by Theorem 1 basic properties of Euler's totient function if* $p$ *is prime then* $\phi(p^k) = p^k - p^{k-1}$*, so* $\phi(7) = 6, \phi(2^4) = 2^4 - 2^3 = 2^3 = 8, \phi(5) = 4$*, so* $\phi(560) = 6 \cdot 2^3 \cdot 4 = 3 \cdot 2^6 = 192$*, so there are* $192$ *integers less than* $560$ *s.t. they are coprime w.r.t. $56*

## Classification of Groups of Unit

**Lemma 2.1.**
*1.* $U(2) \cong \mathbb{Z}_1$
*2.* $U(4) \cong \mathbb{Z}_2$

*Proof.* 1. The only element of $U(2)$ is $1$, so $U(2)$ must be isomorphic to the trivial group, which itself is a cyclic group of order $1$ so it is isomorphic to $\mathbb{Z}_1$.

2. The only elements of $U(4)$ are $1, 3$. Consider a maping $\psi : U(4) \to \mathbb{Z}_2, 1 \mapsto 0, 3 \mapsto 1$. Clearly it is a bijection so it remains to show it is a homomorphism to prove the two groups are isomorphic. Let $a, b \in U(4)$ and consider $\psi(ab)$. There are $4$ cases:
1. $a = b = 1$, then $ab = 1$, then $\psi(ab) = \psi(1) = 0 = 0 + 0 = \psi(1) + \psi(1)$
2. $a = b = 3$, then $ab = 1 \mod 4$, then $\psi(ab) = \psi(1) = 0 \equiv_{\mod 2} 1 + 1 = \psi(3) + \psi(3)$.
3. $a = 1, b = 3$ then $ab = 3$ then $\psi(ab) = \psi(3) = 1 = 0 + 1 = \psi(1) + \psi(3)$
4. $a = 3, b = 1$, but since $U(4)$ and $\mathbb{Z}_2$ are abelian this case reduces to the previous case.

By verifying all cases we conclude $\psi$ is a homomorphism, and it is also bijective, so it is an isomorphism, so $U(4) \cong \mathbb{Z}_2$ via $\psi$.

□

We state without proof two additional results, attributed to Gauss:

**Lemma 2.2.**
*1.* $U(2^n) \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{n-2}}$ *for* $3 \leq n$
*2.* $U(p^n) \cong \mathbb{Z}_{p^n - p^{n-1}}$ *for* $p$ *prime and also* $p \neq 2$

These two lemmas lead to a very powerful result:

**Theorem 3.**
*Let* $U(n)$ *be the group of units of* $n$*, then* $U(n)$ *is isomorphic to an external direct product of groups of integers under modular addition.*

*Proof.* Consider the unique prime factorization of $n$ into $n = \prod_{i=1}^{k} p_i^{\alpha_i}$ where $p_i$ is a prime and $\alpha_i$ is the exponent of the prime in the factorization of $n$ (for example, $80 = 5 \cdot 2^4$ so we have $p_1 = 5, \alpha_1 = 1$ and $p_2 = 2, \alpha_2 = 4$). All $p_i^{\alpha_i}$ terms are relatively prime, so by 2.1 extended Chinese remainder theorem $U(n) \cong U(p_1^{\alpha_1}) \times \cdots \times U(p_k^{\alpha_k})$.

Consider $U(p_i^{\alpha_i})$, there are $4$ cases:
1. $p_i \neq 2$, in which case by 2.2 we have $U(p_i^{\alpha_i}) \cong \mathbb{Z}_{p_i^{\alpha_i} - p_i^{\alpha_i - 1}}$.
2. $p_i = 2$ and $\alpha_i = 1$, in which case by 2.1 we have $U(p_i^{\alpha_i}) = U(2) \cong \mathbb{Z}_1$.
3. $p_i = 2$ and $\alpha_i = 2$, in which case by 2.1 we have $U(p_i^{\alpha_i}) = U(4) \cong \mathbb{Z}_2$.
4. $p_i = 2$ and $3 \leq \alpha_i$, in which case by 2.2 we have $U(p_i^{\alpha_i}) = U(2^{\alpha_i}) \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{\alpha_i - 2}}$

Consider $\times_{i=1}^{k} U(p_i^{\alpha_i})$ which we have shown to be isomorphic to $U(n)$, and recall the following results which apply to any external direct product:
1. ("associativity") $H_1 \times H_2 \times H_3 \cong (H_1 \times H_2) \times H_3 \cong H_1 \times (H_2 \times H_3)$
2. (external direct product preserves isomorphism) If $K \cong H$, then $H \times G \cong K \times G$ and also $G \times H \cong G \times K$.

By these two results, we can replace each $U(p_i^{\alpha_i})$ with either a group of integers under modular addition (cases $1 - 3$), in which case by the second result (external direct product preserves isomorphism) we have

$$U(p_1^{\alpha_1}) \times \cdots \times U(p_i^{\alpha_i}) \cdots \times U(p_k^{\alpha_k}) \cong U(p_1^{\alpha_1}) \times \cdots \times \mathbb{Z}_{t_i} \times \cdots \times U(p_k^{\alpha_k})$$

For some $t_i \in \mathbb{Z}$, or $U(p_i^{\alpha_i}) \cong \mathbb{Z}_2 \times \mathbb{Z}_{2^{\alpha_i - 2}}$, in which case by the second result we have

$$U(p_1^{\alpha_1}) \times \cdots \times U(p_i^{\alpha_i}) \cdots \times U(p_k^{\alpha_k}) \cong$$

$$U(p_1^{\alpha_1}) \times \cdots \times (\mathbb{Z}_2 \times \mathbb{Z}_{2^{\alpha_i - 2}}) \times \cdots \times U(p_k^{\alpha_k}) = (*)$$

Then by the first result ("associativity"), we have

$$(*) \cong U(p_1^{\alpha_1}) \times \cdots \times \mathbb{Z}_2 \times \mathbb{Z}_{2^{\alpha_i - 2}} \times \cdots \times U(p_k^{\alpha_k})$$

Since every $U(p_i^{\alpha_i})$ can be replaced like this, we eventually have $U(n) \cong \times_{i=1}^{w} \mathbb{Z}_{a_i}$ for some $a_1, \ldots, a_w \in \mathbb{Z}$, which completes the proof.

□

This result, while impressive and useful in its own right, is not important for understanding the naive RSA algorithm we discuss in this entry.

### More Euler's Totient Function

**Corollary 3.1** (Euler's totient function for product of coprimes)**.**
*Let $n \in \mathbb{N}$ and $M = \{m_1, \ldots, m_k\}$ be a set of relatively prime integers s.t. $n = \prod_{i=1}^{k}$, then:*
*1.* $|\mathbb{Z}_n^\times| = \prod_{i=1}^{k} |\mathbb{Z}_{m_i}^\times|$
*2.* $\phi(n) = \prod_{i=1}^{k} \phi(m_i)$

*Proof.*
1. Immediately from Theorem 2 since order is preserved under an isomorphism, and $|\times_{i=1}^{n} \mathbb{Z}_{m_i}^\times| = \prod_{i=1}^{n} |\mathbb{Z}_{m_i}^\times|$ by definition of the external direct product and by counting.
2. From $(1)$ and proposition $(1)$ of Theorem 1 basic properties of Euler's totient function.

□

**Theorem 4** (Fermat's little theorem)**.**
*If $p$ is prime and $a$ is not a multiple of $p$, then $a^{p-1} \equiv 1 \mod p$.*

*Proof.* Since $p$ is prime then $|\mathbb{Z}_p^\times| = p - 1$ by Theorem 1 basic properties of Euler's totient function. Consider $\langle a \rangle \leqslant \mathbb{Z}_p^\times$, it is a subgroup of a finite group so by Lagrange's theorem $|\langle a \rangle| | |\mathbb{Z}_p^\times| = p - 1$. Since $\langle a \rangle$ is cyclic then $|a| = |\langle a \rangle|$. By definition of the order of an element, $a^{|a|} = e$. In $\mathbb{Z}_p^\times$ (and any group of integers under modular multiplication) the identity is $1$, so $a^{|a|} \equiv 1 \mod p$, but $|a| | p - 1$ as

we have argued, so $p-1 = n|a|$ for some $n \in \mathbb{Z}$, so $a^{p-1} = a^{n|a|} = (a^{|a|})^n$ by exponent laws of group operation, so $a^{p-1} = e^n = e$, so $a^{p-1} \equiv 1 \mod p$. □

**Theorem 5** (Euler's theorem)**.**
*If $x \in \mathbb{Z}_n^\times$ then $x^{\phi(n)} \equiv 1 \mod n$.*

*Proof.* Since $x \in \mathbb{Z}_n^\times$ and by definition $\phi(n) = |\mathbb{Z}_n^\times|$, then by an immediate corollary to Lagrange's theorem we immediately have $x^{|\mathbb{Z}_n^\times|} \equiv_n 1$. □

We are now ready to describe the RSA encryption algorithm.

## RSA Encryption Algorithm

The RSA encryption algorithm is carried as follows.

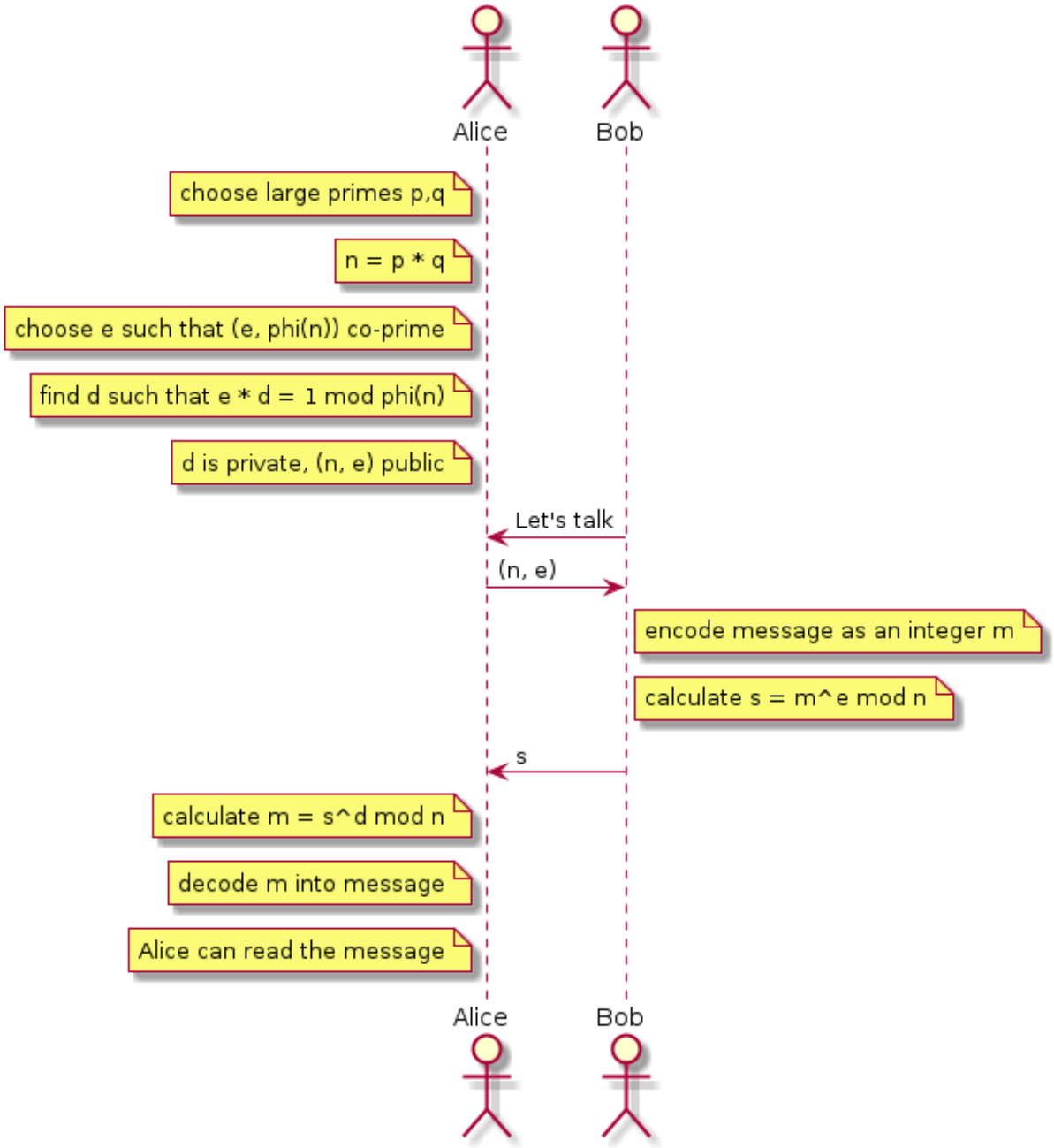
Suppose Alice and Bob wish to establish secure one-way communication from Bob to Alice. Bob will be sending encrypted messages to Alice, and Alice will be establishing the secure channel and decrypting the messages sent by Bob.

1. Pick two very large prime numbers $q, p$.
2. Calculate their product $n = qp$
3. Find some $e \in \mathbb{Z}$ s.t. gcd$(e, \phi(n)) = 1$. We declare $e$ *the encryption exponent*.
4. Find some $d \in \mathbb{Z}$ s.t. $ed \equiv 1 \mod \phi(n)$. $d$ is the *decryption exponent*.
5. $(n, e)$ is the *public key* which can be broadcasted in an unsecure channel, while $d$ is the *public key* which is used to decrypt encrypted messages and should be kept secret.

Next, Bob encrypts a message he'd like to send Alice, as follows:
1. Bob encodes the message into an integer $m$ s.t. $mn$, using an invertible encryption scheme (large message can be broken down into smaller messages to account for the requirement that $m$ be less than $n$)
2. Calculate $s \equiv m^e \mod n$, where $(n, e)$ are the public key pair provided by Alice
3. Bob sends $s$ to Alice

Now, Alice decodes the message:
1. Alice calculates $s^d \mod n$.
2. The result of the previous step is $m$, the encoded message sent by Bob.
3. Alice decodes the message by means of the inverse of the encoding function used by Bob.
4. Alice retrieves the original message.

This process is illustrated in the following diagram:




**Theorem 6** (RSA correctness)**.**
*The RSA encryption algorithm as described above is correct. More specifically:*
*1. Every step is well-defined.*
*2.* $m \equiv s^d \mod n$*, where* $m, s, d$ *are all calculated as described above.*

*Proof.* 1. We go over the algorithm step-by-step:
1. First, two large primes $p, q$ are chosen. By a classical result in arithmetic we know there are infinitely

many primes so we can always find two arbitrarily large primes.
2. Next, we calculate $n = pq$. Note here that $pq$ is also the unique factorization of $n$ by the fundamental theorem of arithmetic. Multiplication is computationally easy.
3. Since $p, q$ are prime then they are coprime, so by 3.1 Euler's totient function for product of coprimes we know that $\phi(n) = \phi(pq) = \phi(p)\phi(q)$. By Theorem 1 basic properties of Euler's totient function we know that $\phi(p) = p-1, \phi(q) = q-1$ since $p, q$ are prime, so $\phi(pq) = (p-1)(q-1)$, which reduces the problem of finding $\phi(pq)$ to a simple multiplication of two known numbers, which is computationally easy.
4. $U(\phi(n))$ is the group of units of $\phi(n)$, and each element of $a \in U(\phi(n))$ satisfies $\gcd(a, \phi(n)) = 1$, so choosing $e$ s.t. $\gcd(e, \phi(n)) = 1$ essentially means choosing $e \in U(\phi(n))$.
5. Since $e \in U(\phi(n))$ and $U(\phi(n))$ is a group, then $\exists! d \in U(\phi(n))$ that is the inverse of $e$, s.t. $ed \equiv 1 \mod \phi(n)$. This $d$ can be found using the extended Euclidean algorithm.
6. Taking $m^e \mod n$ is simply modular exponentiation, which is well-defined.
7. Taking $s^d \mod n$ is well-defined for the same reason as 6.

Thus every step of the algorithm is well-defined.

2. Note that $s^d \equiv_{\mod n} (m^e)^d \equiv_{\mod n} m^{ed}$. Since $ed \equiv 1 \mod \phi(n)$ then $ed = 1 + k\phi(n)$ for some integer $k$, so $m^{ed} = m^{1+k\phi(n)} = m^1 m^{k\phi(n)} = m^1 (m^{\phi(n)})^k$, but by Theorem 5 Euler's theorem we have $m^{\phi(n)} \equiv 1 \mod n$, so $m^1 (m^{\phi(n)})^k \equiv_{\mod n} m^1 1^k \equiv_{\mod n} m$, which is the original message.

□

Since $d$ is unique, the difficulty of cracking an RSA encrypted message given $(n, e)$ is equivalent to that of finding the inverse of $e$ in $U(\phi(n))$, or equivalently finding $d$ s.t. $ed \equiv 1 \mod \phi(n)$. Suppose such $d$ is found by an attacker, then since $ed \equiv 1 \mod \phi(n)$ then $ed = 1 + k\phi(n)$ for some $k$, i.e. the attacker knows some multiple of $\phi(n)$. Note that $\phi(n) = \phi(pq) = (p-1)(q-1)$, so $k\phi(n) = k(p-1)(q-1)$, and also $n = pq$. Given knowledge of a multiple of $\phi(n)$, and knowledge that $n$ is the product of two primes, the problem of finding these two primes becomes computationally easy. We will not show this here, but instead prove a weaker proposition which should provide some intuition as to why that is the case:

**Proposition 1.**
*If $n$ is the product of two primes and $n, \phi(n)$ are known, then factorizing $n$ is computationally easy.*

*Proof.* Recall $\phi(n) = (p-1)(q-1) = pq + 1 - (p+q) = n + 1 - (p+q)$, so $p + q = \phi(n) - (n-1)$. Since $n, \phi(n)$ are known that we can find $p + q$ in via simple subtraction.

Consider the quadratic $(x-p)(x-q)$. On the one hand, its roots are $p, q$. On the other hand, we have

$$(x-p)(x-q) = x^2 - (p+q)x + pq = x^2 - (p+q)x + n$$

Since all coefficients are known, we can use the quadratic formula and find the roots of the polynomial. Since these roots are $p, q$, then we have found the prime factorization of $n$.

□

## Attacks Against RSA

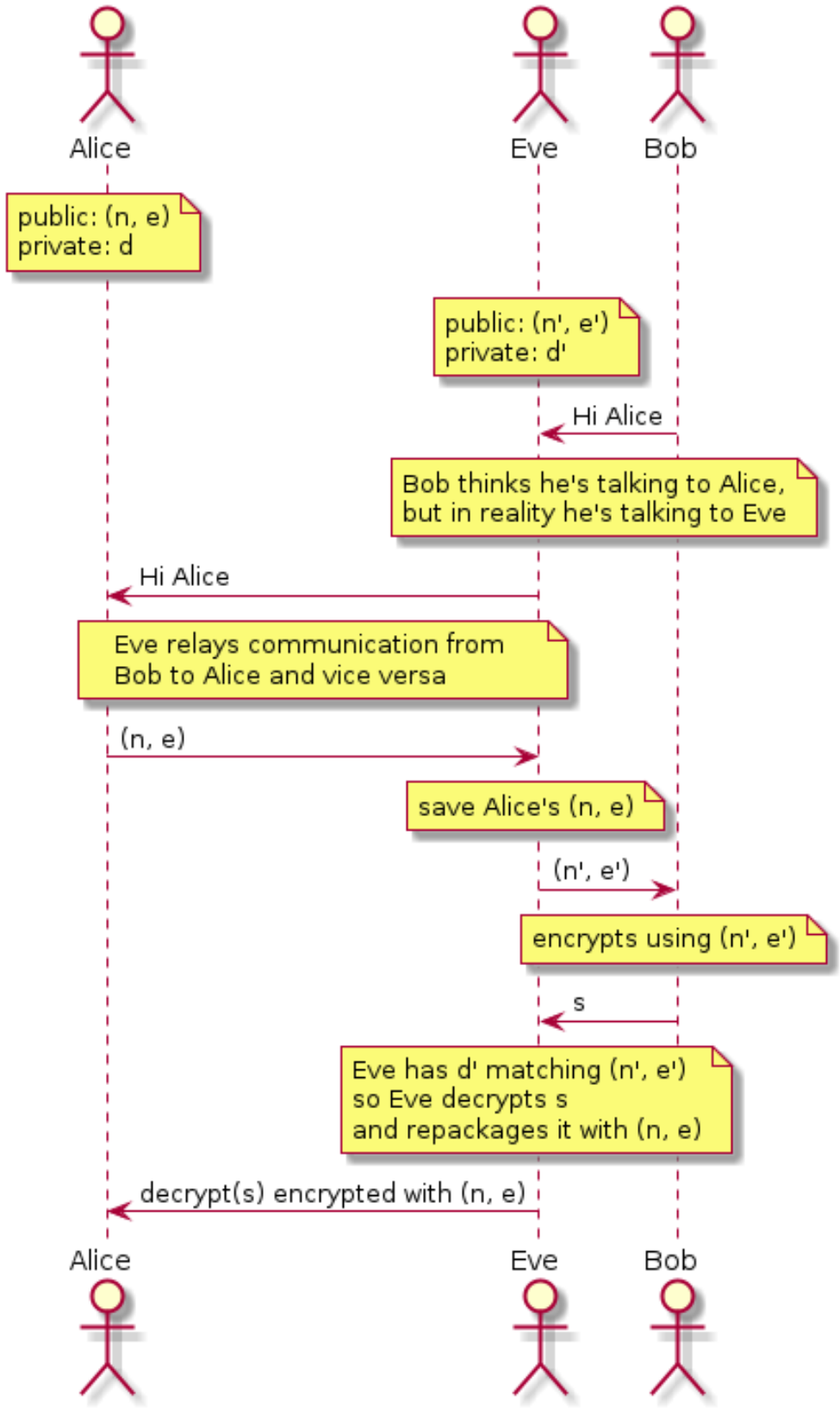
While the security of RSA relies on the difficulty of the integer factorization problem, some properties of RSA make it susceptible to attacks which, if mounted successfully, can crack RSA encrypted messages. In this section we present a very non-exhaustive survey of basic attacks which can be used against RSA.

### Key Distribution Problem

We begin with an issue that is not specific to RSA but is a general problem in cryptography.

Suppose a malicious third-party Eve can not only read all communication between Alice and Bob, but also intercept and modify the information being communicated. In this case, Eve may intercept the initial public key transfer and replace it with their own public key before passing it along to Bob. Then, whenever a message is sent from Bob to Alice (unwillingly) through Eve, Eve will decrypt the message (since the message is actually encrypted using Eve's public key), then re-encrypt he (potentially modified) message and pass it along to Alice.

This type of attack is called a *man in the middle attack*. The following diagram illustrates it:


Alice
Eve
Bob
public: (n, e)
private: d
public: (n', e')
private: d'
Hi Alice
Bob thinks he's talking to Alice,
but in reality he's talking to Eve
Hi Alice
Eve relays communication from
Bob to Alice and vice versa
(n, e)
save Alice's (n, e)
(n', e')
encrypts using (n', e')
s
Eve has d' matching (n', e')
so Eve decrypts s
and repackages it with (n, e)
decrypt(s) encrypted with (n, e)
Alice
Eve
Bob

The issue here is that the initial distribution of the public key was not secure, however for Bob to be able to encrypt messages that only Alice may open, Bob needs to know Alice's public key, which must be communicated to Bob before he may establish secure communication with Alice. This is known as the key distribution problem.

One way to solve the key distribution problem is to transfer new public keys over secure communication with an already-established party. For example, if Dave had already established secure communication with Alice and Bob prior to Eve's man-in-the-middle attack, Alice and Bob may successfully communicate with Dave, so Alice may generate a public key and send it to Dave, and then when Bob wishes to talk to Alice, he would ask Dave for Alice's public key (or ask Dave to verify that the public key he'd been provided matches Alice's public key provided to Dave). Since communication with Dave is secure, Eve can no longer intercept the communication between Alice and Bob successfully.

Of course, this raises the issue of what happens when communication with Dave hadn't already been established before the channel became insecure. This, of course, is a bootstrapping problem. One way to solve it is by using *key servers*, which are agreed-upon servers that serve as a root of trust for key distribution. Key servers serve the same functionality Dave served in the example from the last paragraph.

### Chosen-Ciphertext Attack (CPA)

RSA is deterministic - given an encoded message $m$ and a public-private key pair $((n, e), d)$, $m$ will always be encrypted to the same secret $m^e \mod n$, thus an attacker can guess the content of $m$ and attempt to encrypt it, and if the encrypted message matches the intercepted encrypted message, then the attacker knows that the guess is correct.

This issue is worsened by the fact that, since RSA encryption is performed via modular exponentiation, and since the group of units of $n$ is abelian since modular multiplication commutes, then modular exponentiation commutes with modular multiplication, i.e. $(m_1 m_2)^e \equiv_n m_1^e m_2^e$. Suppose $m_1 = m$, i.e. $s \equiv_n m_1^e$ is the intercepted encrypted secret, so $(m_1 m_2)^e \equiv_n s m_2^e$ for any $m_2$. Since Eve can choose $m_2$ and calculate $m_2^e$ via simple modular exponentiation given $(e, n)$, then Eve can always calculate $(m_1 m_2)^e$ for arbitrary values of $m_2$ (since this is always equivalent to $s m_2^e \mod n$, and $s$ is the intercepted value so it is known, $m_2$ is chosen by Eve and $(e, n)$ are public).

This means that if, for any $m_2$, Eve manages to convince Alice to decrypt $(m_1 m_2)^e$ and reply with the decrypted value $m_1 m_2$, then since $(m_1 m_2)^e \mod n, m_1 m_2 \mod n, m_2$ are all known by the attacker, the attacker can solve for $m_1$ by finding the modular inverse of $m_2$ in $\mod n$.

To solve this issue, randomized *padding* is to $m$ before encryption (i.e. before exponentiation). The padding scheme must also be agreed upon publicly (i.e. the format of the padding, its length, if its

length is random - then the index in the decrypted message of the size of the padding, etc.), however this information in general does not provide an attacker with any meaningful data.

Once randomized padding is introduced, a message should ideally never be encrypted into the same value $s$ twice, so chosen-ciperhtext attacks are rendered irrelevant.

### Attacks via Chinese Remainder Theorem

The Chinese remainder theorem (which we prove in elementary number theory), if $n_1, \ldots, n_k$ are all relatively prime then the system of modular equations of the form $x \equiv_{n_1} a_i$ for $a_1, \ldots, a_k \in \mathbb{Z}$, has a unique solution $\mod \prod_{i=1}^{k} n_i$.

Suppose the same message $m$ is to be sent to different recipients that all share the same encryption exponent $e$ but different $n$ values (this can happen since the only restriction we impose on $e$ is that $\gcd(e, \phi(n)) = 1$, but obviously being relatively prime does not partition the integers into disjoint subsets, so it can be that $\gcd(e, \phi(n_1)) = 1$ and also $\gcd(e, \phi(n_2)) = 1$ for the same $e$).

Suppose the same message is being sent to $k$ recipients with moduli $n_1, \ldots, n_k$. Further suppose that for any $i, j \in [1, k]$, the corresponding prime pairs $(p_i, q_i)$ and $(p_j, q_j)$ are disjoint, i.e. $n_i, n_j$ do not share a common factor. In this case, all of $n_1, \ldots, n_k$ are relatively prime. Suppose all encrypted message $s_i \equiv m^e \mod n_i$ are intercepted, then by the Chinese remainder theorem the system of $k$ equations of the form $m^e \equiv s_i \mod n_i$ has a unique solution $\mod \prod_{i=1}^{k} n_i$.

Now, recall that we require $mn$, so $mn_i$ for every $n_i$, so $m\min\{n_1, \ldots, n_k\}$, so $m^k \min\{n_1, \ldots, n_k\}^k \prod_{i=1}^{k} n_i$.

Suppose now that $e \leq k$, then $m^e m^k$, but also $m^e \prod_{i=1}^{k} n_i$, so we know that the value of $m^e \mod \prod_{i=1}^{k} n_i$ using the Chinese remainder theorem is in fact the real $m^e \in \mathbb{Z}$ (and not another value congruent to it $\mod \prod_{i=1}^{k} n_i$), so now taking the $e^{\text{th}}$ root of $m^e$ in $\mathbb{Z}$, we retrieve the original message $m$.

There are two ways to mitigate this attack:
1. Randomized padding makes it so that the same $m$ is never exponentiated into the same secret, even before taking $\mod n$ (since we always add some padding $p$ before encryption, and $p$ is randomized).
2. Choosing a large encryption exponent $e$, and randomizing the choice of $e$.